

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
In [3]: image = cv2.imread("image.jpg")

if image is None:
    print("Image not found. Please check the image path.")
else:
    print("Image loaded successfully!")
    print("Image shape:", image.shape)
```

Image loaded successfully!
Image shape: (1422, 800, 3)

```
In [4]: image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8, 6))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Original Image



```
In [5]: gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(8, 6))
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()
```

Grayscale Image



```
In [6]: blurred = cv2.GaussianBlur(gray, (5, 5), 0)

plt.figure(figsize=(8, 6))
plt.imshow(blurred, cmap="gray")
plt.title("Gaussian Blurred Image")
plt.axis("off")
plt.show()
```

Gaussian Blurred Image



```
In [7]: threshold_value = 127

_, global_thresh = cv2.threshold(
    blurred,
    threshold_value,
    255,
    cv2.THRESH_BINARY
)

plt.figure(figsize=(8, 6))
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")
plt.show()
```

Global Thresholding



```
In [8]: otsu_value, otsu_thresh = cv2.threshold(
        blurred,
        0,
        255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

print("Otsu's Threshold Value:", otsu_value)

plt.figure(figsize=(8, 6))
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")
plt.show()
```

Otsu's Threshold Value: 130.0

Otsu's Thresholding



```
In [9]: adaptive_thresh = cv2.adaptiveThreshold(
        blurred,
        255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY,
        11,
        2
    )

plt.figure(figsize=(8, 6))
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")
plt.show()
```

Adaptive Thresholding



```
In [10]: _, binary = cv2.threshold(
          blurred,
          0,
          255,
          cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
        )

plt.figure(figsize=(8, 6))
plt.imshow(binary, cmap="gray")
plt.title("Binary Image for Watershed")
plt.axis("off")
plt.show()
```

Binary Image for Watershed



```
In [11]: kernel = np.ones((3, 3), np.uint8)
```

```
opening = cv2.morphologyEx(  
    binary,  
    cv2.MORPH_OPEN,  
    kernel,  
    iterations=2  
)
```

```
In [12]: sure_bg = cv2.dilate(opening, kernel, iterations=3)
```

```
In [13]: dist_transform = cv2.distanceTransform(  
    opening,  
    cv2.DIST_L2,  
    5  
)  
  
_, sure_fg = cv2.threshold(  
    dist_transform,  
    0.5 * dist_transform.max(),  
    255,  
    0  
)
```

```
sure_fg = np.uint8(sure_fg)
```

```
In [14]: unknown = cv2.subtract(  
        sure_bg,  
        sure_fg  
    )
```

```
In [15]: _, markers = cv2.connectedComponents(sure_fg)  
  
        markers = markers + 1  
  
        markers[unknown == 255] = 0
```

```
In [16]: markers = cv2.watershed(image, markers)  
  
        watershed_result = image_rgb.copy()  
  
        watershed_result[markers == -1] = [255, 0, 0]  
  
        plt.figure(figsize=(8, 6))  
        plt.imshow(watershed_result)  
        plt.title("Watershed Segmentation")  
        plt.axis("off")  
        plt.show()
```

Watershed Segmentation



Observation

The boundaries between different regions are identified. Watershed is particularly useful for separating objects that are touching each other.

Step 10: K-Means Clustering

K-Means is a clustering-based segmentation technique.

Instead of using only intensity, we can use the RGB color information.

First reshape the image.

```
In [17]: pixel_values = image_rgb.reshape((-1, 3))  
  
pixel_values = np.float32(pixel_values)  
  
print("Pixel data shape:", pixel_values.shape)
```

Pixel data shape: (1137600, 3)

```
In [18]: k = 3  
  
kmeans = KMeans(
```

```
n_clusters=k,  
random_state=42,  
n_init=10  
)  
  
labels = kmeans.fit_predict(pixel_values)  
  
centers = np.uint8(kmeans.cluster_centers_)
```

```
In [19]: segmented_pixels = centers[labels]  
  
segmented_image = segmented_pixels.reshape(image_rgb.shape)  
  
plt.figure(figsize=(8, 6))  
plt.imshow(segmented_image)  
plt.title("K-Means Segmentation")  
plt.axis("off")  
plt.show()
```

K-Means Segmentation



Observation

The image is divided into different regions according to their color similarity.

Step 11: Compare All Segmentation Techniques

Now display the results together.

```
In [21]: import matplotlib.pyplot as plt

plt.close("all")
print("Ready!")
```

Ready!

```
In [23]: plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(watershed_result)
plt.title("Watershed")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(segmented_image)
plt.title("K-Means")
plt.axis("off")

plt.tight_layout()
plt.show()
```



14. Observations

The following observations can be written in your practical file:

Global Thresholding successfully separates foreground and background when illumination is relatively uniform. Otsu's Thresholding automatically determines a suitable threshold and eliminates the need for manual threshold selection. Adaptive Thresholding performs better under uneven illumination because it calculates local thresholds. Watershed Segmentation is effective for separating touching or overlapping objects. K-Means Clustering divides an image into regions based on color similarity. The quality of segmentation depends strongly on the image characteristics, noise level, illumination, and complexity of the background. No single segmentation technique is optimal for every type of image.

15. Result

The different image segmentation techniques—Global Thresholding, Otsu's Thresholding, Adaptive Thresholding, Watershed Segmentation, and K-Means Clustering—were successfully implemented using Python, OpenCV, NumPy, Matplotlib, and Scikit-learn. Their segmentation results were visualized and compared.

In []: